Inteligencia Artificial

ENRIQUE PELÁEZ Ph.D.

Escuela Superior Politécnica del Litoral – ESPOL

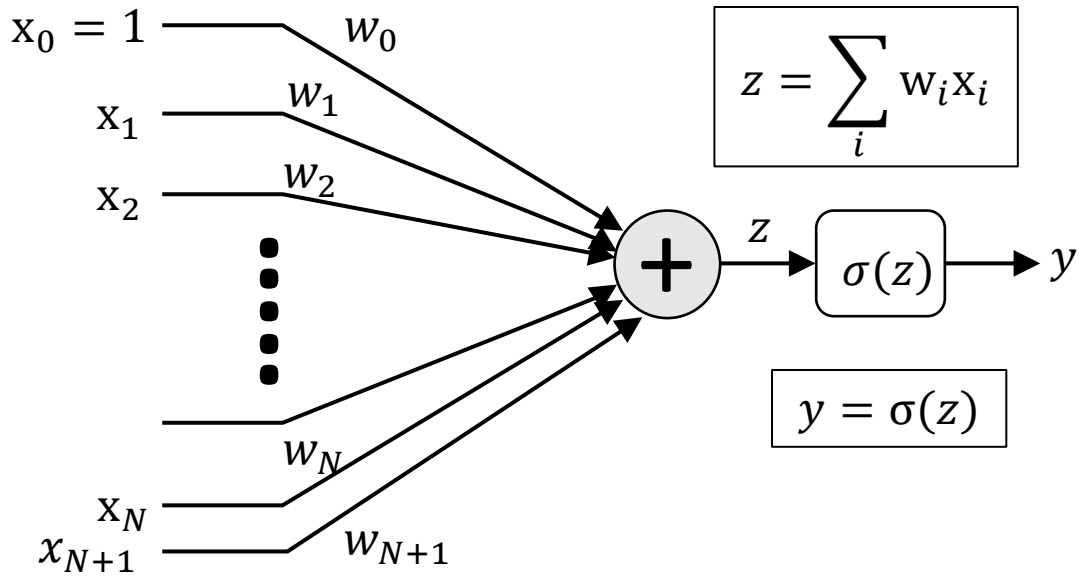
Facultad de Ingeniería en Electricidad y Computación – FIEC

Clase # 10 - PAO 2026 – II

Agenda

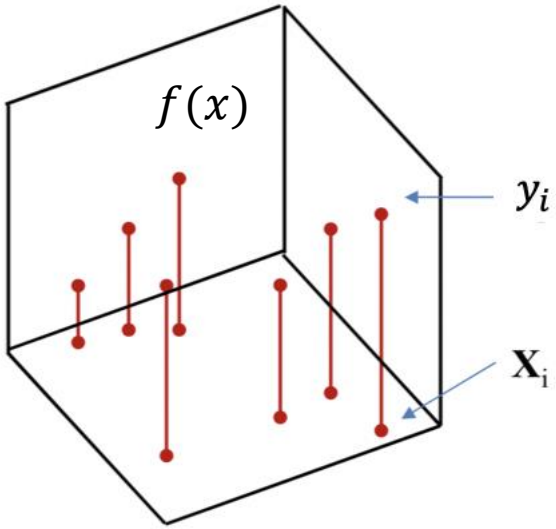
1. Las formas de entrada, salida, funciones de umbral y de error
2. Notación vectorial en las redes MLP
3. Algoritmo de entrenamiento con GD
   3.1 Forward propagation
   3.2 Corrección del error – Back propagation
4. El Jocabian
5. Introducción a las redes Neuronales Convolucionales – CNN

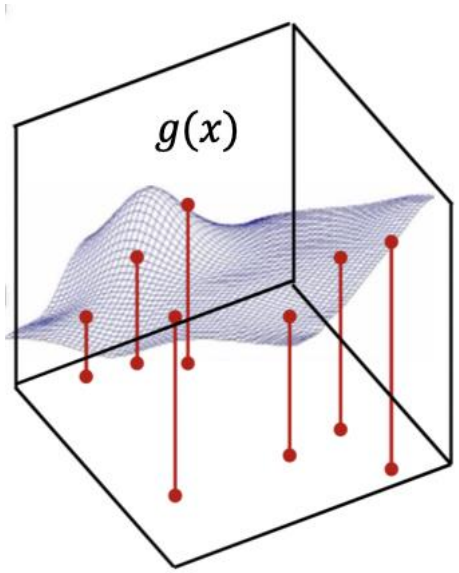
Aprendizaje en red



- El bias también se puede ver como el peso de otro componente de entrada que está fijo en 1
- Notar que: Si el bias no se menciona de manera explícita, siempre está. Recuerde que cada perceptron siempre tiene una entrada adicional (bias) que está fija en 1

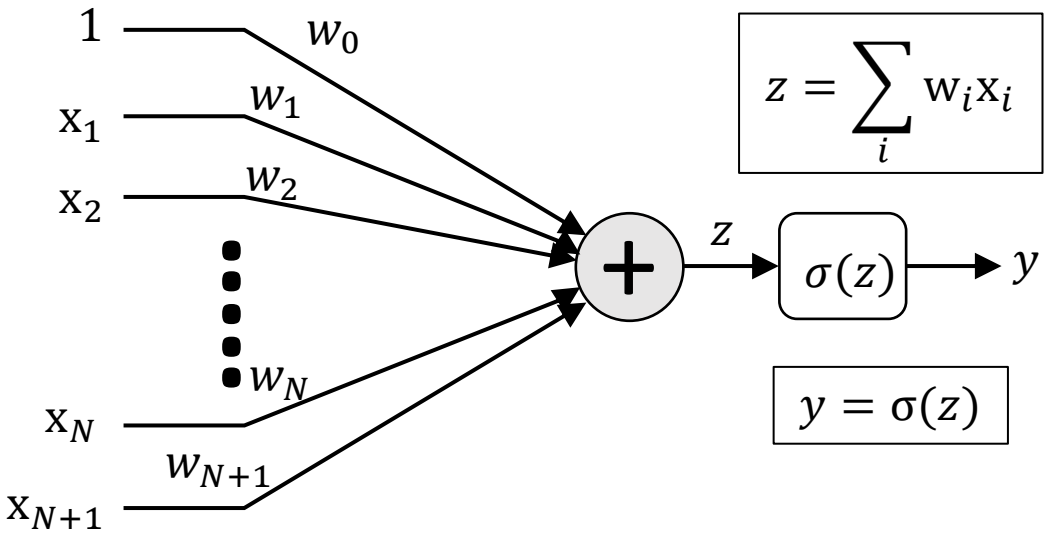
Aprendizaje de la función $g(X)$





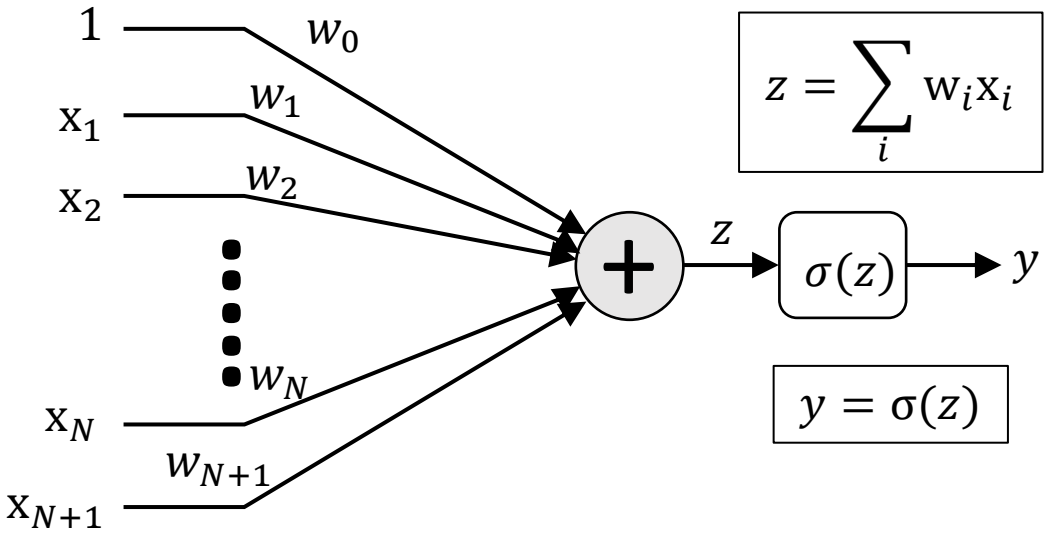
- Buscamos aprender la función $g(X_i)$ completamente a partir de pocos ejemplos….los ejemplos de entrenamiento

Cómo aprendemos la función $g(X)$?



- y es una función que depende de z; $\sigma(Z)$
- z es una función que depende de w_i y x_i
- Esto quiere decir que podemos calcular los cambios en la salida y, cuando ocurren pequeños cambios en la entrada x o en los pesos w
- El mecanismo que nos permite determinar esos cambios, de manera iterativa, es el gradiente descendiente !

Aprendizaje: Perceptron con activación derivable

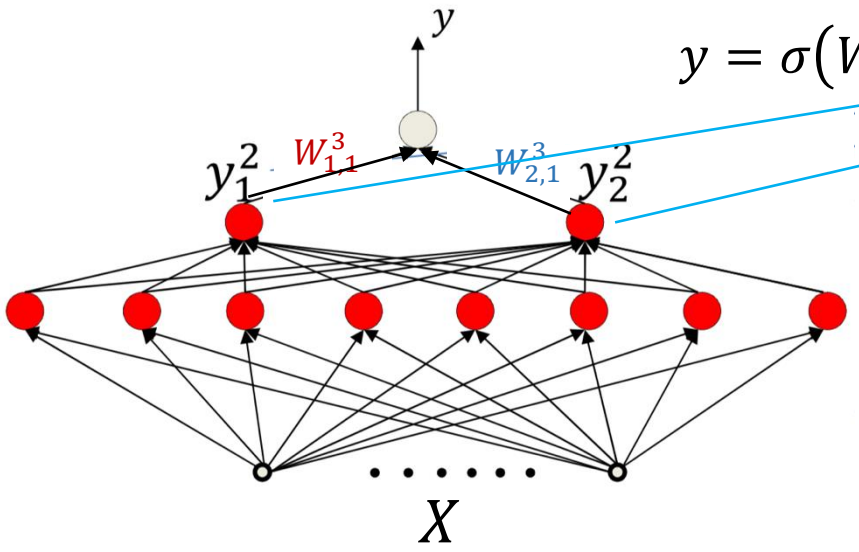


$$\frac{dy}{dw_i} = \frac{dy}{dz}\frac{dz}{dw_i}$$

$$\frac{dy}{dX_i} = \frac{dy}{dz}\frac{dz}{dX_i}$$

- Estas derivadas nos dicen cuan sensible es una función con respecto a las variables que la definen; es decir, cómo un pequeño cambio de las variables impacta al valor total de la función
- Durante el aprendizaje queremos ajustar los pesos, cuando se producen pequeños cambios en la función que define el error total

Aprendizaje: Red MLP derivable



$$y = \sigma\left(W_{1,1}^3 y_1^2 + W_{2,1}^3 y_2^2\right)$$

y = salida total de la red

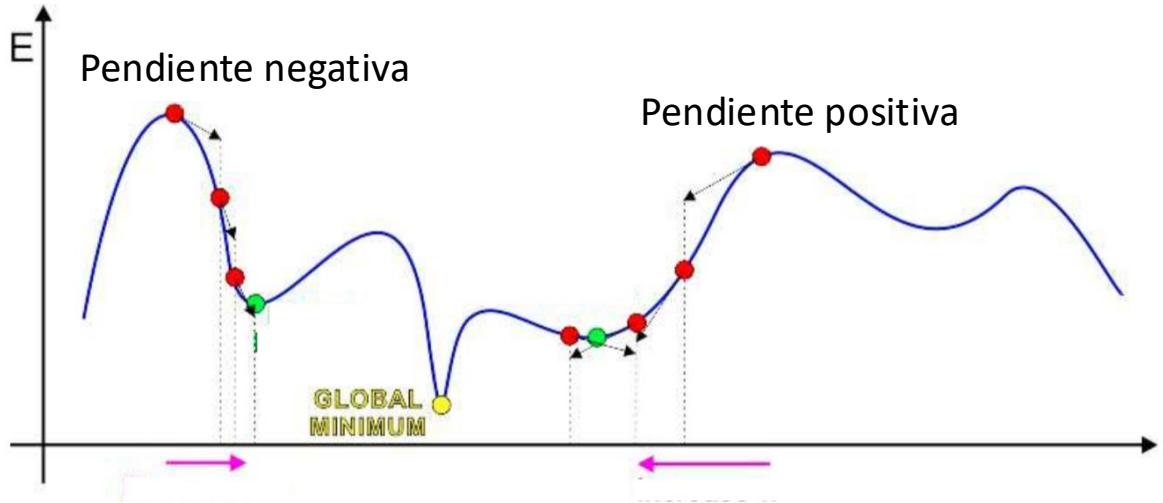
$W_{i,j}^k$ = peso de la conexión de la unidad i^{th} de la capa k^{th} a la unidad j^{th} de la capa $(k+1)^{th}$

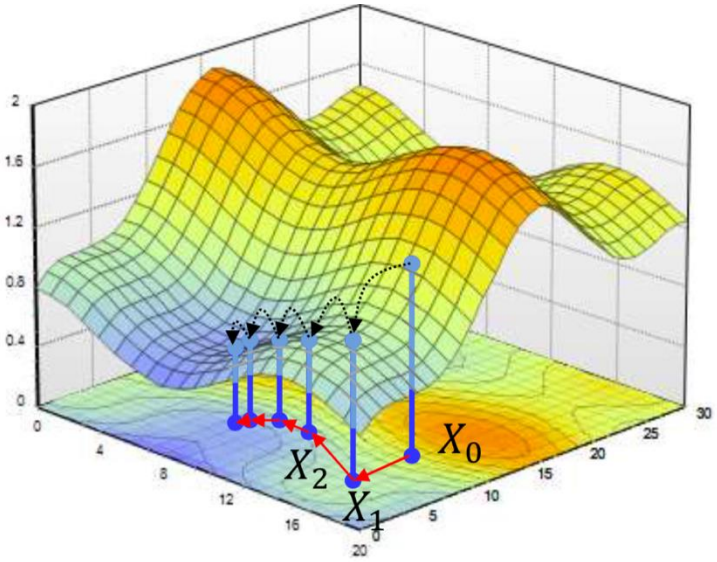
y_i^k = salida de la unidad i^{th} de la capa k^{th}

$\sigma(\,)$ = función de activación o umbral (derivable c.r.a w, x)

- Cada perceptron es derivable c.r.a sus pesos (incluyendo el peso del "bias") y "entradas"
- En Backprop se calcula la derivada de la función que define el error con respecto a cada peso w; así, determinamos cuán sensible es la función de error con respecto a cada uno de estos pesos
- Esto nos ayuda a determinar cuánto debemos cambiar cada parámetro w durante el descenso del gradiente; al final obtenemos el modelo que mejor se ajusta a los datos de entrenamiento

Aprendizaje: Gradiente descendiente

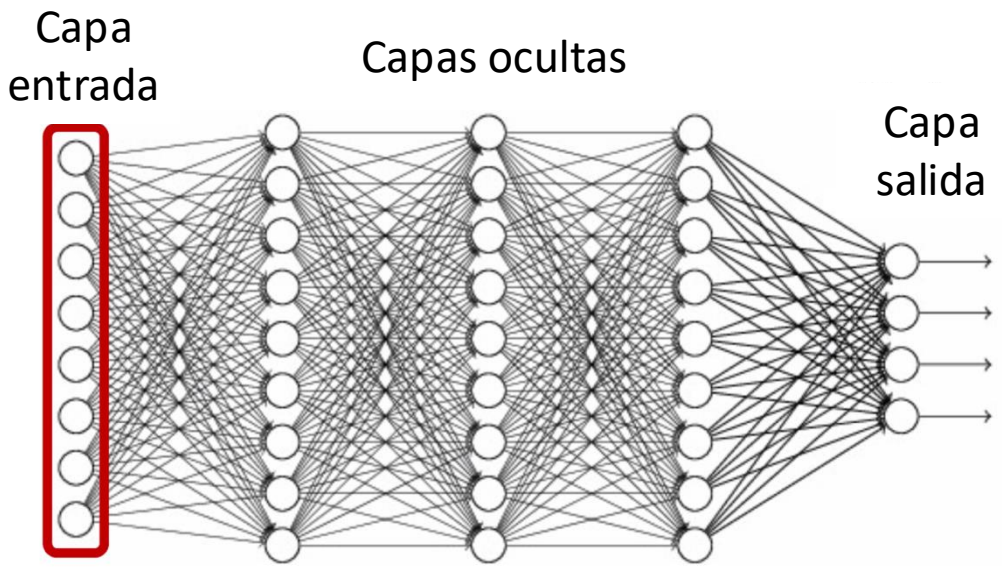




Esto es gradiente descendiente

- Solución iterativa
  - Empezar en un punto inicial (aleatorio)
  - Encontrar el punto en que cambia la dirección, para disminuir el error
    - Esto es, encontrar la derivada de la función:
      - Una derivada positiva, moverse a la izquierda disminuye el error
      - Una derivada negativa, moverse a la derecha disminuye el error
  - Cambiar la dirección en ese punto

Aprendizaje: La entrada



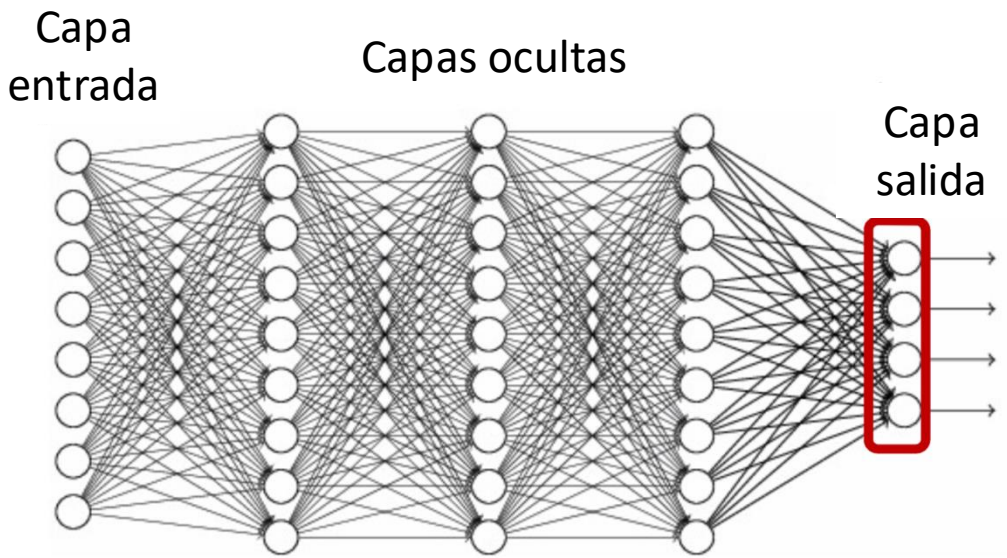
- Vector de números
  - Inclusive un valor escalar; tamaño del vector de entrada 1
  - Vector con valores de pixeles
  - Vector con valores de voz
  - Vector con valores reales representando texto
  - Vector con valores representando la salida de sensores, Otros..

Aprendizaje: La entrada - Tensores

Escalar: 3 Vector: $\begin{bmatrix}1\\2\\3\end{bmatrix}$ Matriz: $\begin{bmatrix}1 & 2\\3 & 4\end{bmatrix}$ Tensor: $\begin{bmatrix}[1 & 2] & [3 & 4]\\[5 & 6] & [7 & 8]\end{bmatrix}$

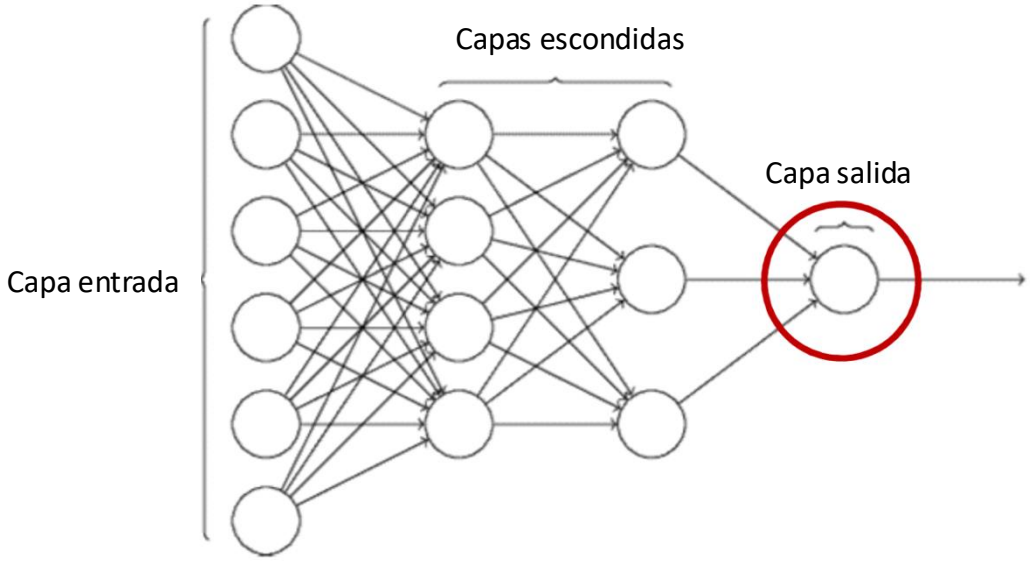
- Tensores
  - Escalar: Tensor de dimensión 0 (0D). Por lo tanto, tiene 0 ejes y es de rango 0
  - Vector: Tensor de una sola dimensión (1D). Un vector tiene 1 eje y es de rango 1
  - Matriz: Tensor de rango 2, o 2 ejes, organizada en filas y columnas; técnicamente es un Tensor de 2 dimensiones (2D)
  - Tensor 3D (y mayor dimensionalidad): Si bien, técnicamente, todas las estructuras anteriores son tensores válidos, los tensores generalmente se refieren a las matrices de 3 o más dimensiones

Aprendizaje: La salida



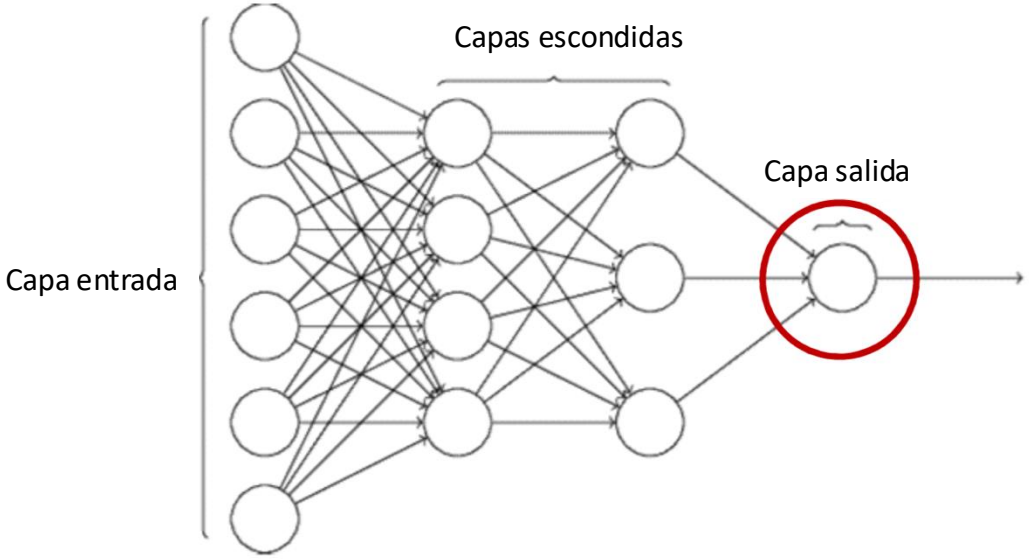
- La salida generalmente es un valor real
  - Si la salida es escalar: Una neurona
  - Salida es un vector: Tantas neuronas como se requiera en la salida
  - $d = [d_1 d_2 \dots d_L]$

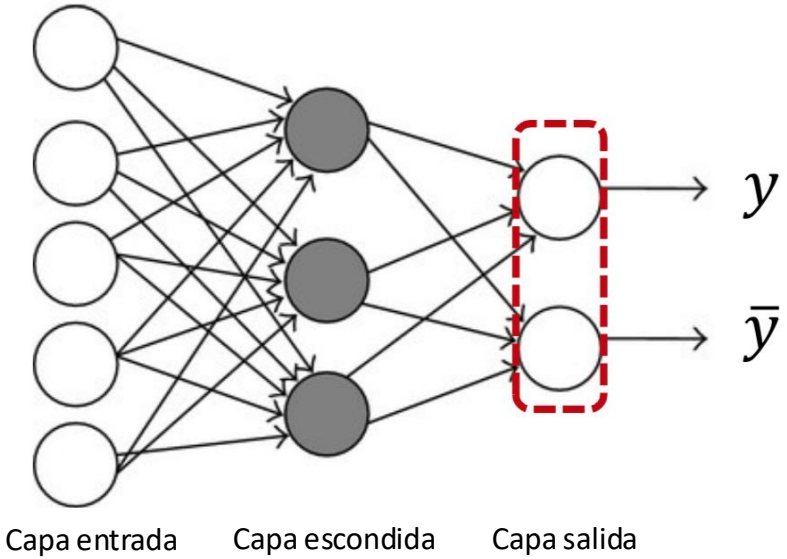
Aprendizaje: La salida



- Si la salida es binaria, con una neurona es suficiente
  - Si = 1
  - No = 0

Aprendizaje: La salida





- O puede utilizarse 2 neuronas independientes: una representa la salida deseada, la otra no
  - Si = [1 0]
  - No = [0 1]

Aprendizaje: Salida multi-clase– representación one-hot

- Considere una red que debe distinguir si una entrada es: perro, mono, flor, sombrero o gato
- Podemos representar este conjunto con el siguiente vector:

  [perro mono flor sombrero gato]
- Para entradas de cada una de estas clases, la salida deseada es:

  perro: $[1\ 0\ 0\ 0\ 0]^T$

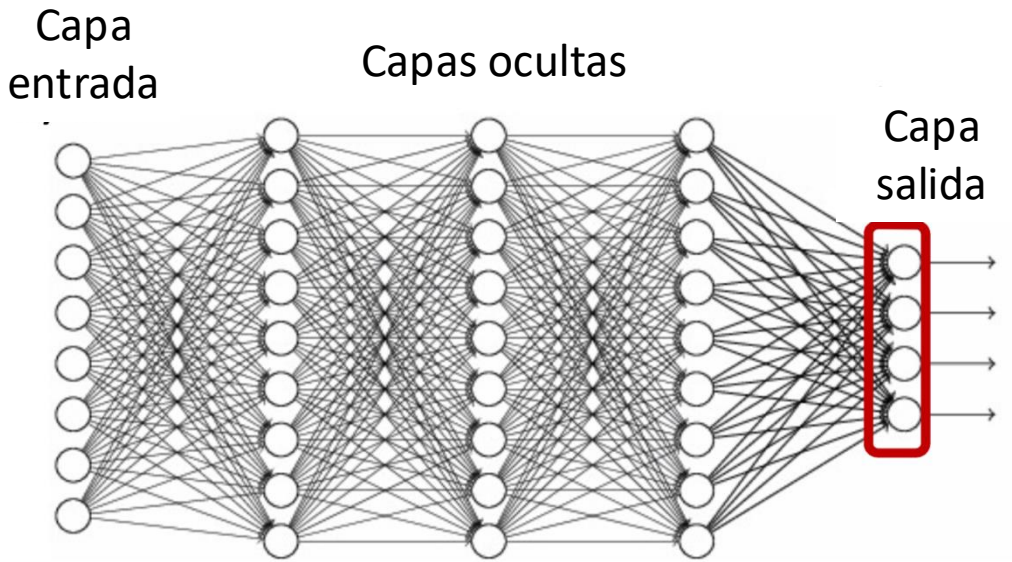
  mono: $[0\ 1\ 0\ 0\ 0]^T$

  flor: $[0\ 0\ 1\ 0\ 0]^T$

  sombrero: $[0\ 0\ 0\ 1\ 0]^T$

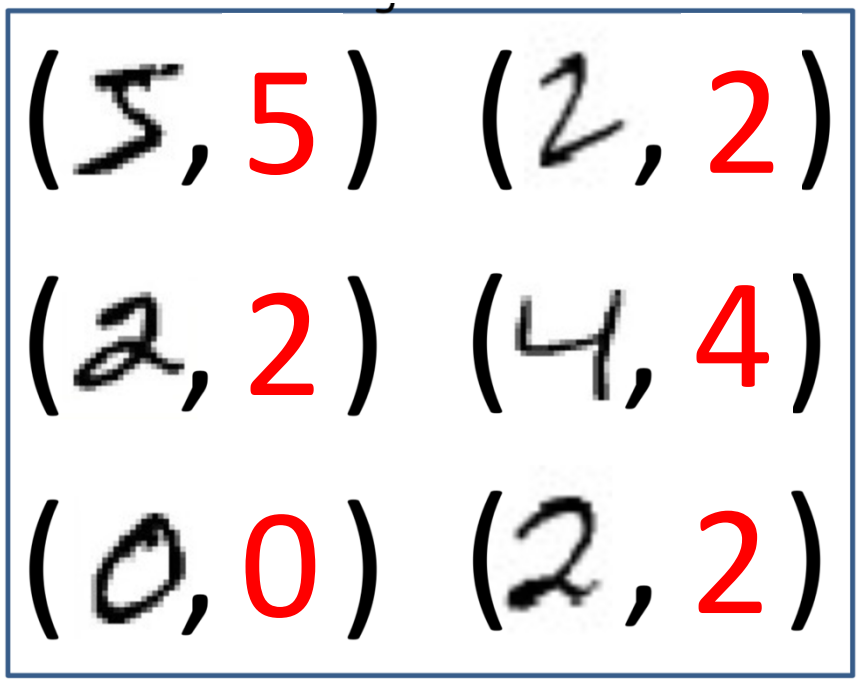
  gato: $[0\ 0\ 0\ 0\ 1]^T$
- Para una entrada de cualquier clase, tendremos un vector de salida de cinco dimensiones con cuatro ceros y un 1 en la posición de la clase correspondiente
- Este se conoce como one-hot encoding

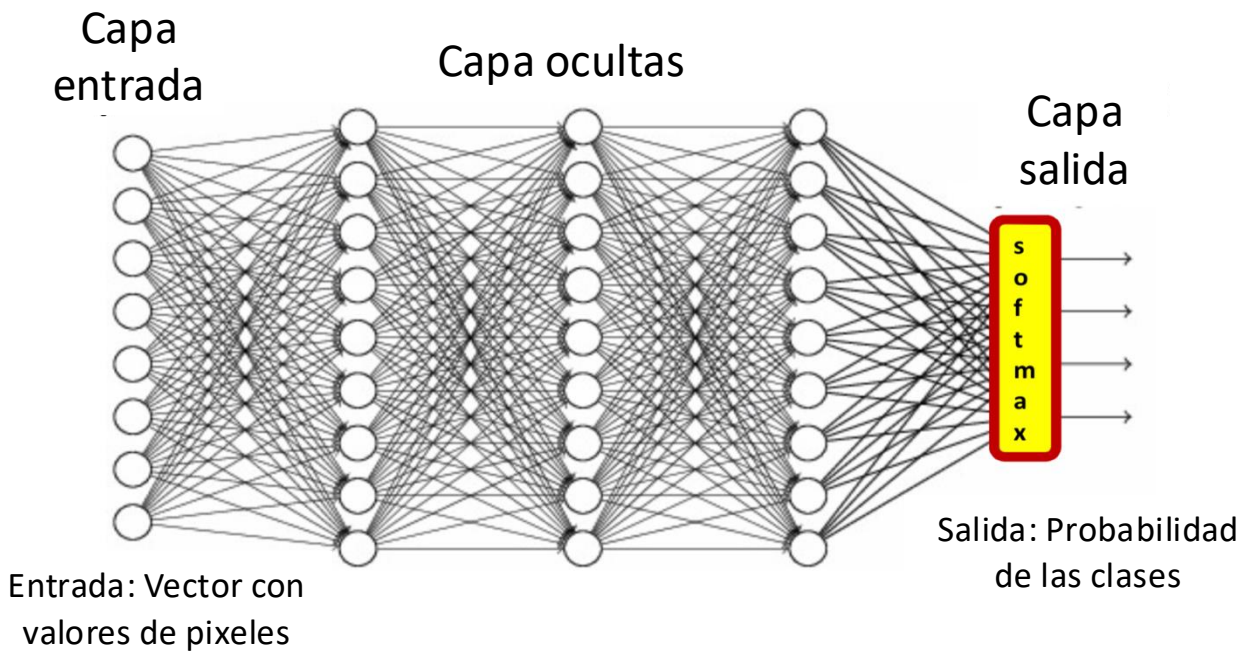
Aprendizaje: Salida en las redes multi-clase



- Para un clasificador de clases múltiples con L clases, la representación one-hot tiene L salidas binarias
  - Un vector binario L-dimensional
- La salida de la red es binaria (L-1 ceros y un 1 en el lugar correcto)
- Si la salida es un vector de probabilidades, como en el caso de una función sigmoidal
  - L probabilidades que suman 1

Aprendizaje: Clasificación multi-clase





- Con muchos ejemplos (positivos y negativos) para entrenamiento
  - Aprender todos los pesos de la red, para determinar qué digito es

Aprendizaje: Qué es $f()$

- Softmax
  - Las unidades Softmax se usan para distribuciones de salida Multi-clase, se usa para una distribución de probabilidad sobre una variable discreta con n valores posibles, como una generalización de la función Sigmoide que representa la distribución de probabilidad sobre una variable binaria:

$$\mathrm{softmax}(\boldsymbol{z})_i = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$

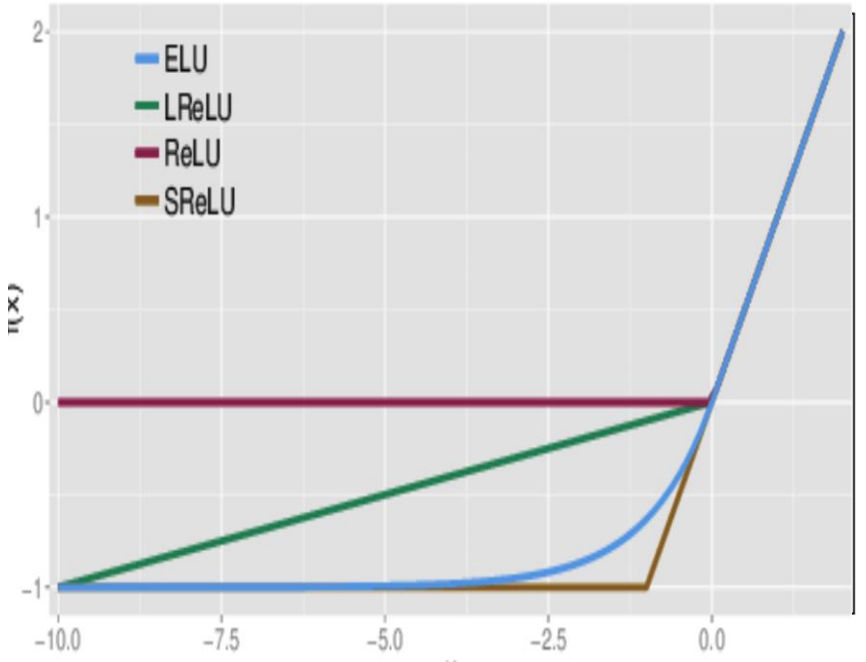
Aprendizaje: Qué es $f()$

Las opciones para unidades ocultas son:

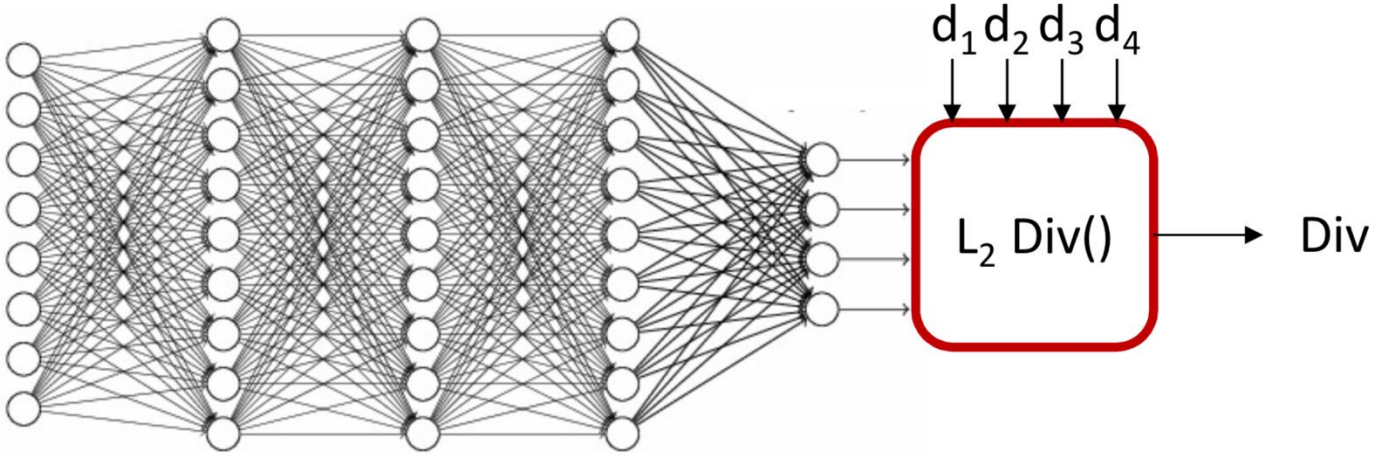
- Unidades lineales rectificadas (Rectified linear units – Relu)

$$g(z) = \max\{0, z\}$$

- Hay algunas generalizaciones de Relu:
  - Rectificación de valor absoluto
  - ReLU exponencial - ELU
  - ReLU con fugas – LReLU: Leaky ReLU
  - ReLU paramétrico – SReLU: Shifted ReLU



Aprendizaje: Ejemplos de funciones $div()$

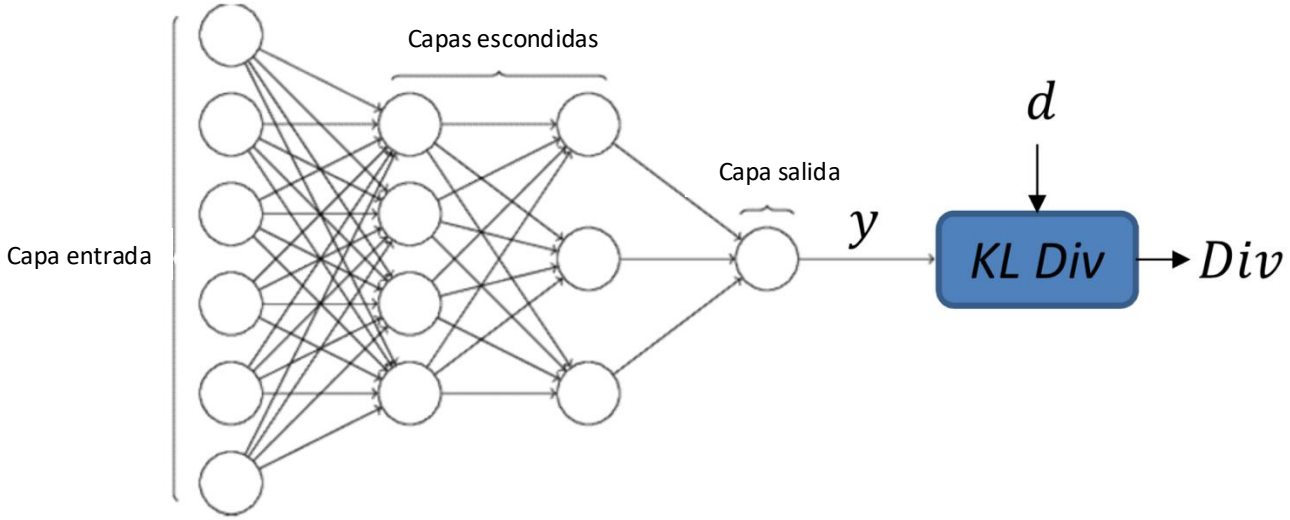


- El cuadrado de la distancia Euclideana entre la salida real y la deseada, conocida como la divergencia L_2 (es popular)

$$Div(Y, d) = \frac{1}{2}\|Y - d\|^2 = \frac{1}{2}\sum_i (y_i - d_i)^2$$

Aprendizaje: Ejemplos de funciones $div()$

- Clasificador binario

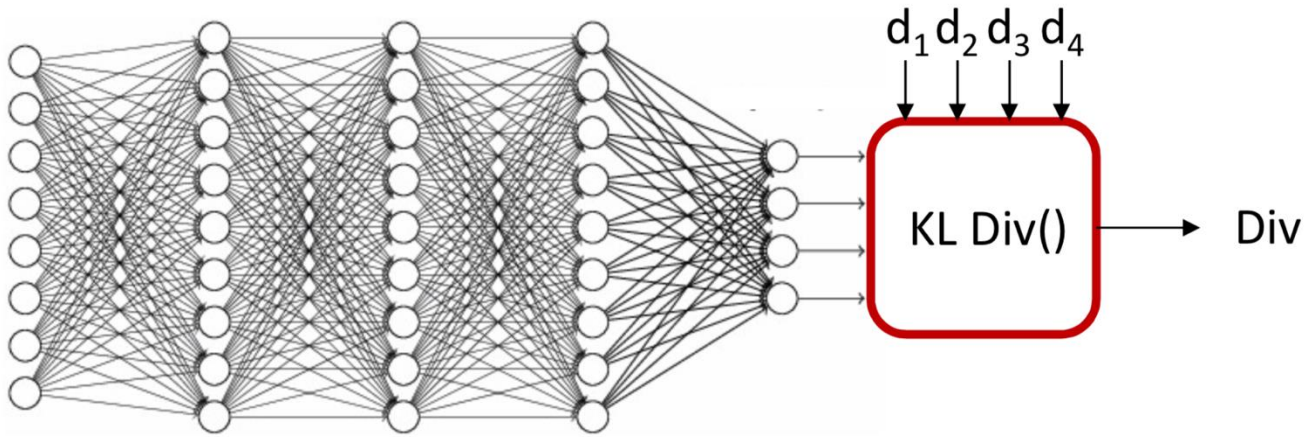


- La salida de un clasificador binario $Y \in [0,1], d\ es\ 0/1.$
- Una función común es el cálculo de la entropía (cross entropy) entre la distribución probabilística de la salida real $[Y, 1-Y]$ y la probabilidad de la salida deseada $[d, 1-d]$

$$Div(Y,d) = -dlogY - (1-d)log(1-Y)$$

Aprendizaje: Ejemplos de funciones $div()$

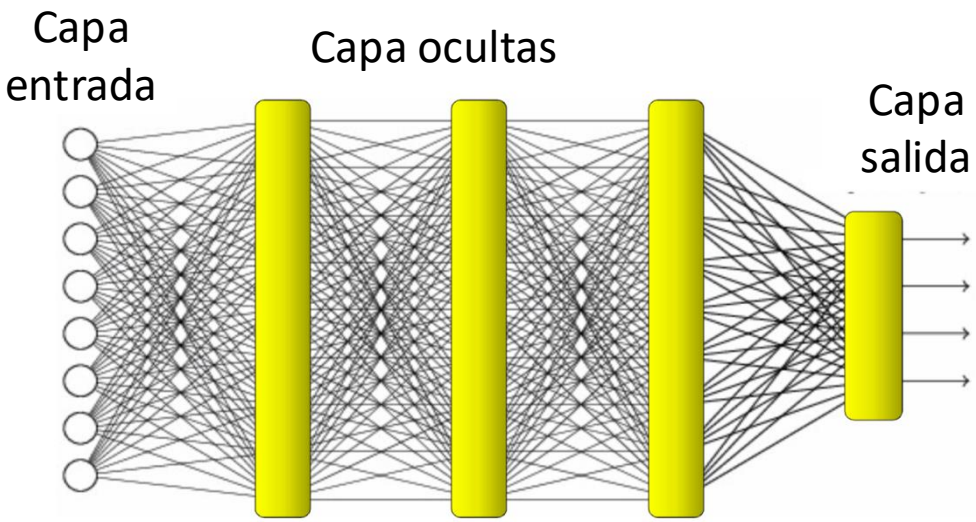
- Clasificador multi-clase



- La salida deseada es: un vector "one-hot" [00 ... 1 ... 000] con un 1 en la posición c^{th} (para clase c)
  - La salida es la distribución probabilística $[y_1, y_2, \ldots]$
  - La entropía cruzada entre la salida multiple deseada y la salida real:

$$Div(Y, d) = -\sum_i d_i log\ y_i$$

Aprendizaje: Arquitectura típica de una red



$$z_j^{(1)} = \sum_i w_{ij}^{(1)} x_i$$

$$z_j^{(k)} = \sum_i w_{ij}^{(k)} y_j^{(k-1)}$$

$$y_j^{(k)} = f_k\left(z_j^{(k)}\right)$$

- Cada capa de perceptrones se la puede considerar como un vector de activación